Table Of Contents

1.0 - Device Overview 3
1.1 - Features 3
1.2 - General Description 4
1.3 - Block Diagram 5
2.0 - Device Characteristic Table 6
2.1 - System Characteristics 6
2.2 - Micro-Controller Interface (MSP432E401Y) 6
2.3 - Stepper Motor and Driver 7
2.4 - VL53L1X Time-of-Flight Sensor 7
3.0 - Detailed Description 9
3.1 - Distance Measurements 9
3.2 - Data Verification and Processing 10
3.3 - Visualization 11
4.0 - System Operation, User Manual, and Output Validation 12
4.1 - Application 12
4.2 - Clock Configuration 12
4.3 - User Manual 13
4.4 - Expected Output 14
5.0 - Limitations 18
6.0 - Circuit Schematic 20
7.0 - Flow Chart 21
8.0 - References 23

1.0 - Device Overview

1.1 - Features

[1] Texas Instruments MSP432E401Y Microcontroller:
● ARM Cortex-M4F processor core with integrated Floating-Point Unit (FPU) for a plethora of efficient numerical computations
● Configured bus speed of 26 MHz (supports up to 120 MHz system clock)
● Supports I2C communication up to 3.33 MHz for sensor interfacing
● UART communication configured at 115200 bps for reliable serial data transmission
● On-board LEDs used for system status indication (measurement, pre-scan, return state)
● GPIO-based push buttons (PM0, PM1) used for user-controlled scanning and data transmission
● Approximate Cost: ~$ 70.00CAD

[2] VL53L1X Time-of-Flight (ToF) Distance Sensor:
● Laser-based distance measurement up to approximately 4 meters - can be configured to different modes
● Communicates with the microcontroller via I2C protocol (typically 400 kHz operation)
● Provides high-resolution distance measurements for spatial mapping
● Operates at 3.3 V from micro
● Approximate Cost: ~$20.00 CAD

[3] Stepper Motor (Rotational Scanning Mechanism):
● Controlled via GPIO pins (PH0–PH3) using a predefined stepping sequence
● Enables precise angular rotation (11.25° increments per measurement step)
● Supports both clockwise (CW) and counterclockwise (CCW) motion
● Operates at 5.0 V using an external driver interface UNLV board
● Approximate Cost: ~$0.00 CAD, homemade.

[4] Serial Communication System:
● I2C protocol used for communication between the microcontroller and ToF sensor via VL53L1X API
● UART protocol used for transmitting collected scan data to a PC
● PySerial library used on the PC side for serial data acquisition and parsing
● Enables real-time monitoring and post-processing of scan data
● Software Cost: Free (open-source libraries)

[5] 3D Visualization and Data Processing:
● Python-based application used to process and interpret collected scan data

● Converts polar coordinate measurements (angle and distance) into Cartesian coordinates
● Matplotlib used to generate 3D images of the scanned environment
● Enables reconstruction of indoor spaces such as hallways across multiple scan positions
● Software Cost: Free (open-source Python libraries)

1.2 - General Description

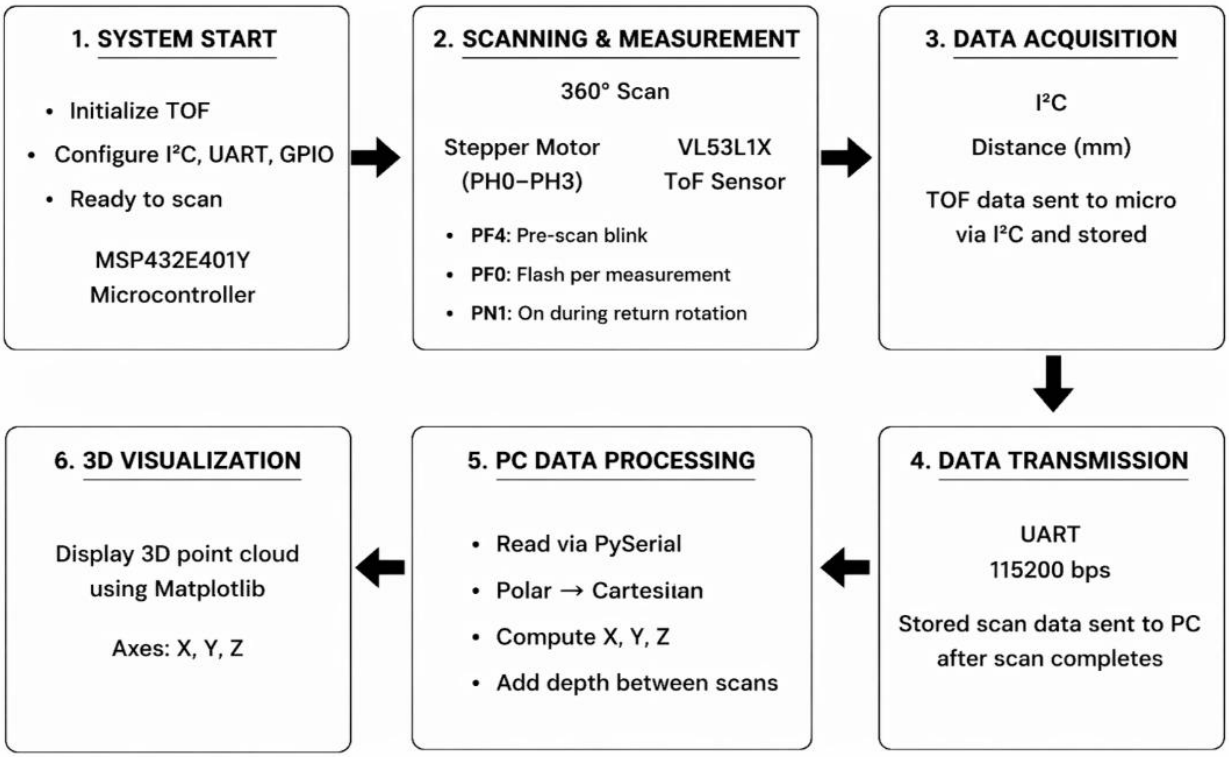
This embedded system is a cost-effective solution for users to collect and visualize 3D spatial data in a controlled manner. The system consists of a VL53L1X Time-of-Flight (ToF) sensor, which uses infrared pulses to measure how long it takes to return to the sensor after reflecting off objects in the vicinity, allowing the sensor to measure distances accurately. A Texas Instruments MSP432E401Y microcontroller is used to control the sensor through I2C interface. The ToF Sensor is mounted on a stepper motor, which is controlled using GPIO Port H (PH0-PH3). The stepper motor, when activated by the external button PM0, moves the sensor through a full clockwise rotation of 360° in 32 discrete steps with an increment of 11.25° and pauses briefly on each step to provide a stable position for the sensor to take a distance measurement.

The distance measurement taken by the ToF sensor is signalled by the ToF sensor's flashing LED (PF0), which also provides live feedback to the user that data is being acquired. The LED (PF4) will flash for the duration just before a scan; this indicates to the user that a scan is about to occur. After the 360° counter-clockwise rotation, the stepper motor will return to the home (0°) position, while LED (PN1) is turned on to indicate to the user that the system is in the return phase and will prevent a snag in the cable.

Distance measurements are accumulated from multiple scans via I2C and retained in memory on the microcontroller. When the operator presses the second button PM1, these saved scan records will be sent to the PC using UART serial communication operating at 115200 bps. The PC uses a Python script utilizing the PySerial library that interprets each measurement in the incoming data stream and transforms polar coordinates (angle and distance) to three-dimensional (x,y,z) coordinates via trigonometric relationships. Multiple scans may be added together by progressively translating the location between scans to create an impression of depth.

The modified data is displayed on the computer screen in 3D as a point cloud via Matplotlib that allows the operator to view a three-dimensional image of what has been scanned, in real time. The MSP432E401Y reaches all system functions such as timing, motor control, sensor communications and data flow between devices using the ARM Cortex-M4F processor for optimal performance and synchronized operation of the entire system

1.3 - Block Diagram



This block diagram stands as a representation of the real-time processing steps the ***Embedded LiDAR Mapping System*** undergoes in real time.

2.0 - Device Characteristic Table

2.1 - System Characteristics

Feature	Description
UART Transmission	115200 bps, 8 data bits, no parity, 1 stop bit
I2C Transmission	100 kbps (I2C0_MTPR = 12)
Data Points per Scan	32 samples (11.25° resolution)
Maximum Scans	3
Steps per Revolution	2048
Steps per Measurement	64 steps (11.25°)
Measurement Step Delay	4 ms
Return Step Delay	3 ms
Debounce Time	20 ms
ToF Timeout	1200 ms
Valid Distance Range	20 mm – 4000 mm
System Clock	26 MHz bus speed
PC Software	Python (PySerial, Matplotlib)

2.2 - Micro-Controller Interface (MSP432E401Y)

Feature	Description
PM0 (Push-Button)	Initiates scans
PM1 (Push-Button)	Transmits stored data
PM0 + PM1	Clears all stored scan data
PJ0 (On-board Button)	Pause / Resume system operation
PJ1 (On-board Button)	Toggles scan direction (CW ↔ CCW)
PF0 (LED – Measurement Indicator)	Measurement Status
PF4 (LED – Scan/Transmission Indicator)	UART Tx
PN1 (LED – Return Indicator)	Return-Home Signal
UART Interface	Serial communication to PC
I2C Interface	Communication with the ToF sensor

2.3 - Stepper Motor and Driver

Feature	Description
IN1	PH0
IN2	PH1
IN3	PH2
IN4	PH3

Drive Method	4-step full-step sequence
Sequence Pattern	(0x0C, 0x06, 0x03, 0x09)
Supply Voltage	5 V

2.4 - VL53L1X Time-of-Flight Sensor

Feature	Description
I2C Address	0x29
Communication Protocol	I2C
Measurement Output	Distance in millimetres
Measurement Method	Photon travel time / 2 * c
Valid Measurement Range	20 mm – 4000 mm
Timeout Handling	Reinit on Timeout
Error Handling	Error Code 0

3.0 - Detailed Description

3.1 - Distance Measurements

ToF (Time of Flight) measurements are obtained by the VL53L1X time-of-flight (ToF) sensor by emitting a laser pulse and measuring how long it takes for the laser to return after bouncing off an object. Since the speed of light is constant, you can calculate distance using:

$$D = Photon\ Travel\ Time/2 * c$$

The VL53L1X sensor executes the steps necessary to perform ToF measurements internally; it emits a photon signal, detects the return signal and processes the signal to get a digital distance value based on the optical signal received. The value (in millimeters) is sent to the MSP432E401Y microcontroller through an I2C interface.

The distance measurement process begins after the system has been initialized and configured and will use a polling method for measuring rather than generating an interrupt when the measurement is complete. During each step of measuring distance, the microcontroller will poll for the sensor's data-ready status until it is ready to read the distance value from the sensor, establishing a synchronization condition between the mechanical positioning of the sensor before the acquisition of valid distance data. A timeout mechanism has been implemented to ensure that if 1200ms elapses without receiving a valid distance measurement; there will no longer be an indefinite block by re-initializing the sensor, ensuring it operates reliably.

Scanning occurs through a nested looping structure whereby 32 unique angles will be produced by getting a full rotation (360 degrees) at 11.25-degree increments. After moving to the location of each angle (64 steps to move from one angle position to the next), the stepper motor will stop rotating so that it can rest (mechanical stabilization) before making a measurement. Upon making the measurement, the measured distance will first be validated to determine if it is within the 20 mm to 4,000 mm limits; anything outside the limits will be recorded as a zero. Each successful measurement will be stored in memory in two 2D arrays - one array for the scan number (up to 3) and the second for the sample number (32) for the scan will provide enough space to store a total of 96 individual data points. During the measurement phase of the process, data will be streamed out through UART while saving the data structure for later transmission.

These steps will continue until 32 measurements have been completed for the scan, at which time the system will turn back around and prepare to either continue scanning or transmit the data it has collected.

3.2 - Data Verification and Processing

When acquiring distances, the VL53L1X sends a range status code indicating the reliability of each individual measurement provided by the sensor. In this application, any non-zero range status as an erroneous measurement by storing/communicating the deemed errant measurement as a relative distance of zero. The range codes are generated by the internal signal processing of the VL53L1X and allow for the identification of potential problems such as weak reflection, excessive noise, or timing issues. It is important to understand the range status codes as they will provide the reason(s) as to why the application has rejected a specific data point and how environmental or system conditions affected the accuracy of the measurement.

0 → Valid Measurement
1 → Sigma Fail (High Noise / Uncertainty)
2 → Signal Fail (Weak Return Signal)
3 → Out of Bounds / Min Range Fail
4 → Hardware Fail / Phase Error
5 → Range Valid but Wrapped / Phase Ambiguity
6 → Processing Fail
7 → Unknown / Other Error

The MSP432 microcontroller sends distance data it has measured to a PC (personal computer) using UART (Universal Asynchronous Receiver/Transmitter) Protocol. The data is then received by the PySerial (Python Serial) library, which formats it into numbers that represent scan number, angle and distance in millimetres. The original polar coordinates of the measurements will then be converted into cartesian coordinates on a personal computer (PCs) by using trigonometric identities and relationships. By performing this calculation on PCs higher precision floating-point calculations can be used due to the capabilities of computers. After converting to cartesian coordinates, the data can then have its position updated for every increment of measurement between each scan, making it possible to combine successive slices of data into a 3D Point Cloud of the environment.

x = displacement, $y = d\cos(\theta)$, $z = d\sin(\theta)$
d - represents the distance measured.
Θ - represents the angular position of the sensor.
Ex: d = 100 mm and theta = 30 degrees, then x = 0 mm, y = 100 * cos(30) = 86.6 mm, and z = 100 * sin(30) = 50 mm.

x - displacement of 10cm each time to ensure moving in “3D”.

3.3 - Visualization

The Python implementation of the visualization process utilizes serial communication through a specific port(e.g. COM4) between the computer and microcontroller. The connection is established in PySerial at 115200 bps to provide continuous reception of data received from the microcontroller via the serial port.

To provide data to the visualization, you must first press the PM1 button on the microcontroller. This action will cause the microcontroller to send all scan data that is stored within its system. Therefore, the creation of the visualization will be based on fully developed and structured measurements, as opposed to receiving live data with only a fraction of the information. Once the PM1 button has been pressed and the sending of data begins, the python program will continuously read each line of data, parse it, and extract the scan number, angular position and distance values. Once the scan data has been parsed, the data will be processed, converted into Cartesian coordinates and stored for visualization.

After all scans have been gathered, the Python program will generate a three-dimensional scatter plot using the matplotlib package. This scatter plot will represent the environment around the robot as accurately as possible. Once all data has been transmitted to the program, the serial connection(s) on COM4 will be closed to release the port number so that it may be used again in the future. Lastly, the completed visualization will be displayed to the user in an interactive 3D window using the matplotlib package. The 3D window will allow the user to view and manipulate the display of the 3D representation of the reconstructed spatial configuration as they desire.

4.0 - System Operation, User Manual, and Output Validation

4.1 - Application

LiDAR has the ability to measure distances using an emitted laser pulse that sends out a laser beam which travels to an object, then returns to the sensor unit and measures the time taken for the return trip. These characteristics allow for the generation of highly detailed 3D images of landscapes, and thus are powerful for both earth and space applications.

For example, the OSIRIS-REx mission run by NASA utilized the OSIRIS-REx Laser Altimeter (OLA) for mapping the surface of the asteroid Bennu. The OLA sent laser pulses to Bennu's surface and measured the time it took for the laser to return back to Earth. OLA built a highly detailed, 3-dimensional representation of the surface of Bennu. The highly detailed representation allows scientists to locate the best spots for sampling and to better understand the surface of Bennu for navigation and collecting samples.

LiDAR is also used for earth-based applications in mapping and monitoring natural environments. In forestry and environmental science, it can penetrate through tree canopies to measure the density of vegetation, estimate the biomass of trees, and produce highly accurate terrain surface models. Using these data, scientists can assess ecosystem health, the rate of deforestation, and the occurrence of natural disasters such as landslides and flooding.

4.2 - Clock Configuration

$$f_{vco} = \frac{f_{xtal}}{(Q+1)\,(N+1)}\,(MINT + \frac{MFRAC}{1024})$$

Where f_{vco} is used in:

$$f_{BUS} = \frac{f_{vcol}}{SYSDIV + 1}$$

Default Configurations:

Q = 0, N = 4, MFRAC = 0 (reduces jitter), f_{xtal} = 25MHz

Changed Configurations:

MINT = 104

Therefore: $f_{vco} = 520\,000\,000$ Hz
$SYSDIV = 19$
Therefore: $f_{BUS} = 26$ MHz

4.3 - User Manual

1. Install the Hardware

a. Attach the MSP432E401Y microcontroller through USB to the PC
b. Attach the VL53L1X Time-of-Flight sensor (I2C SDA = PB3, SCL = PB2) along with power (3.3V) and ground connections
c. Attach the stepper motor controller to port H pins (PH0-PH3)
d. Make sure all pushbuttons (PM0, PM1, PJ0, PJ1) are easy to reach

2. Find the COM Port

a. Open Device Manager and expand "Ports (COM & LPT)"
b. Find "XDS110 Class Application/User UART."
c. Write down what COM port is obtained by opening Device Manager (Example: COM4)

3. Update the Python Script

a. Update com-port in script:
Serial.Serial*('COMX', 115200)*
b. Change scan parameters as required (displacement per scan and number of scans)

4. Flash the Firmware to the Microcontroller

a. Open Project in Keil µVision
b. Build and download firmware to MSP432E401Y
c. Confirm that the board powered up correctly (LED Flash)

5. Start up the system

a. Execute the visualization program (it will wait for data coming in)
b. In order to start up the entire system, press the microcontroller's reset button

6. Scan your surroundings

a. To complete a full 360 degree sweep, press the PM0 button
b. The system will complete the following steps:
- Illuminate PF4 LED briefly before the scan (pre-scan indication)
- Rotate the motor and take 32 measurements

- Flash PF0 LED at each distance measurement
- Automatically return to the starting position after the scan
- Keep PN1 LED ON during the return rotation

c. After each scan is complete, move the entire system forward (e.g., ~10 cm) and press PM0 to perform the next scan

7. Other controls

a. To pause/resume system operation, press the PJ0 button

b. To toggle switch between clockwise and counter-clockwise scanning motion, press the PJ1 button

c. By pressing and holding both the PM0 and PM1 buttons, all scan information will be erased from memory

8. Transmitting / Generating Data Visualization

a. After completing all of the scans, press the PM1 button

b. The system will transmit all of the scan data that is saved in the memory through UART

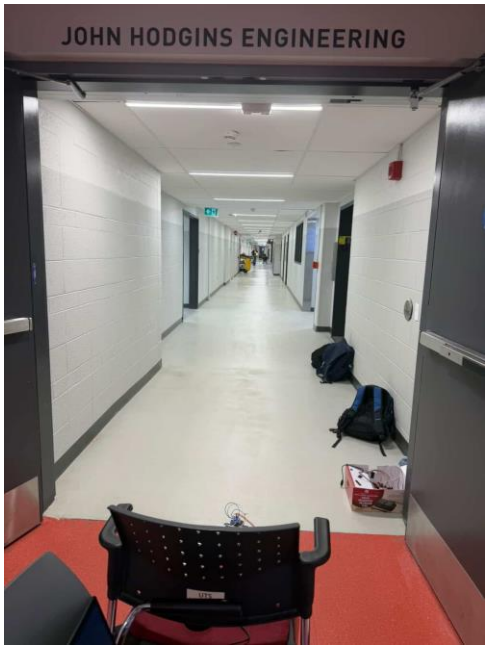
c. The PF4 LED will flash momentarily to indicate that the scan data has been transmitted

d. The Python script will take the data just received and create a three-dimensional plot mapping out the measurement values

e. The completed interactive visualization will appear immediately after the data has been fully plotted

4.4 - Expected Output

Figure 1.0 - Scanned Space



The following image is of the JHE building located on McMaster Campus. It is on the second floor in the HATCH wing.

(Location F)

Backpacks, chairs and any other obstacles were removed during the scan.

Figure 2.0 - Apparatus of the Embedded LiDAR Mapping System

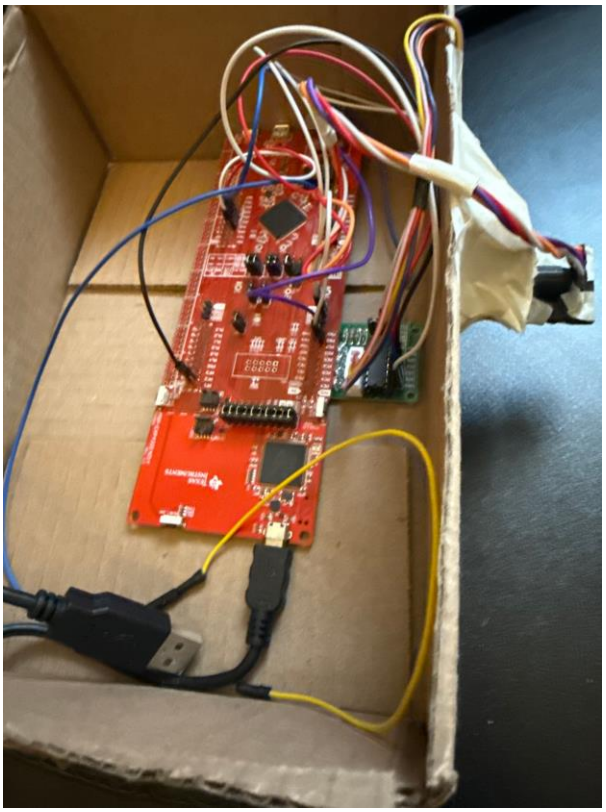


Figure 3.0 - Scanned Plot
Side View

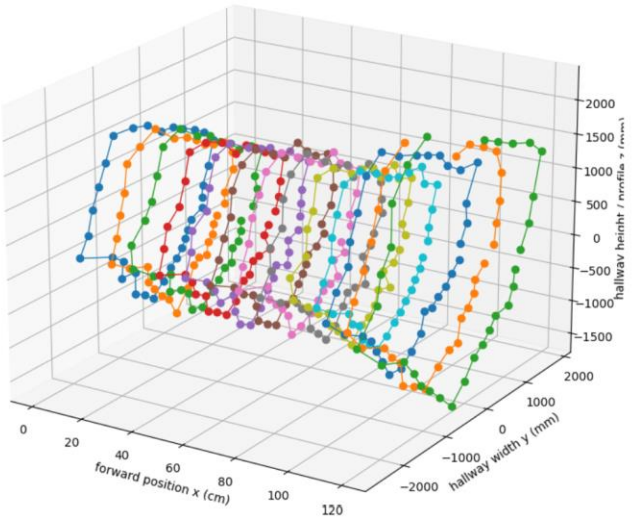
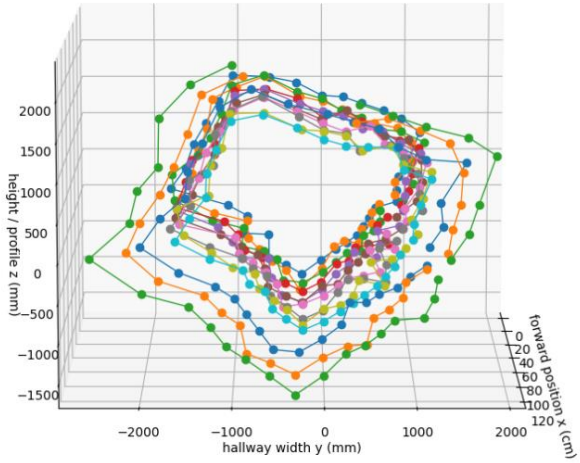


Figure 4.0 - Scanned Plot
Front View



The expected results of the LiDAR scanning system are validated by comparing the original location of the scan with the 3D model produced from it. The location assigned to the scan was Location F, the second floor of JHE.

Figure 1.0 shows an image of the physical corridor that was actually used for the data collection, which can be used as a reference for validating accuracy. Figure 2.0 shows the scanning setup used, which included a microcontroller, a ToF sensor, and an assembly of steppers to move at different angles. Figures 3.0 and 4.0 show the created 3D model, including both front and side views.

The results of the scan show that the LiDAR scanning system has been able to accurately collect and represent the geometry of the corridor that was scanned. The 3D model accurately depicts the width of the scanned corridor, with several areas that become narrower or wider along the length of the scanned corridor. In addition to representing the width of the corridor, the model also captured many smaller objects in the corridor itself, such as recycling bins. The data collected from this LiDAR scanning system meets a very close approximation to the physical environment that was scanned, which demonstrates that this system produces an accurate spatial representation of the scanned environment, therefore confirming the LiDAR scanning system's ability to accurately map a building.

5.0 - Limitations

Maximum Quantization Error

Digital distance measurements from the VL53L1X are represented as discrete numbers or digits. That sensor will provide distance measurements as a 16-bit (unsigned) integer that has a resolution of 1 mm, meaning that the minimum possible measurable distance change is quantized to 1 mm. This quantization causes the system to have finite precision limitations.

The maximum quantization error of a system can be estimated by employing the formula below:

$$Max\ Quantization\ Error\ = \frac{Max\ Reading}{2^{\#ADC\ bits}}$$

$$Max\ Quantization\ Error\ = \frac{4000}{2^{16}}$$

$$Max\ Quantization\ Error\ =\ 0.0610\ mm$$

However, the VL53L1X's effective resolution is actually only **± 0.5 mm** due to the 1 mm quantization limit. As a result of this limitation, the overall measurement accuracy of the sensor for indoor mapping applications will be significantly lower than what could potentially be achieved with very high numerical precision as provided by the digital representation of the readings. Some error may accumulate over time due to multiple scans that may contain small amounts of quantization error during creation of the 3D models from those scans.

UART Communication Rate

The MSP432E401Y microcontroller uses 115200 bps as a UART baud rate to establish reliable communication between itself and PCs; this was verified via experimentation as a reliable method of transferring data without error since both devices had their own clocks running at the same speed. Moreover, having this fixed connection between two devices also serves as a limitation because it does not allow for more than one high-speed (greater than) connection at a time and creates additional issues when trying to transfer large quantities of data (such as multiple scans) in a timely manner and add additional lag into the overall system response time and visualization time of the incoming scanned data.

Floating Point and Trigonometric Limitations

MSP432E401Y microcontroller is built upon an ARM Cortex-M4F core with single precision FPU which allows for high efficiency when executing floating-point arithmetic including trigonometric calculations. However, due to the limitations of single precision arithmetic (32-bits), when used for calculating very small angular increments or values that require a high

degree of numerical accuracy, rounding errors can occur. Over time these errors accumulate when applying polar to Cartesian coordinate conversions. This can cause extremely minor spatial distortion in 3-D reconstruction.

To overcome this limitation, all trigonometry and coordinate conversion calculations are done on the computer instead of the microcontroller. The Python mathematics library and the NumPy library both use double precision (64-bits) floating-point arithmetic and provide much greater numerical precision than single precision. Thus, by separating data acquisition (on the MCU) from computational processes (on the PC), the time to process data has no effect; therefore, the system can have efficient real-time operation yet high precision in the last 3-D reconstructed image.

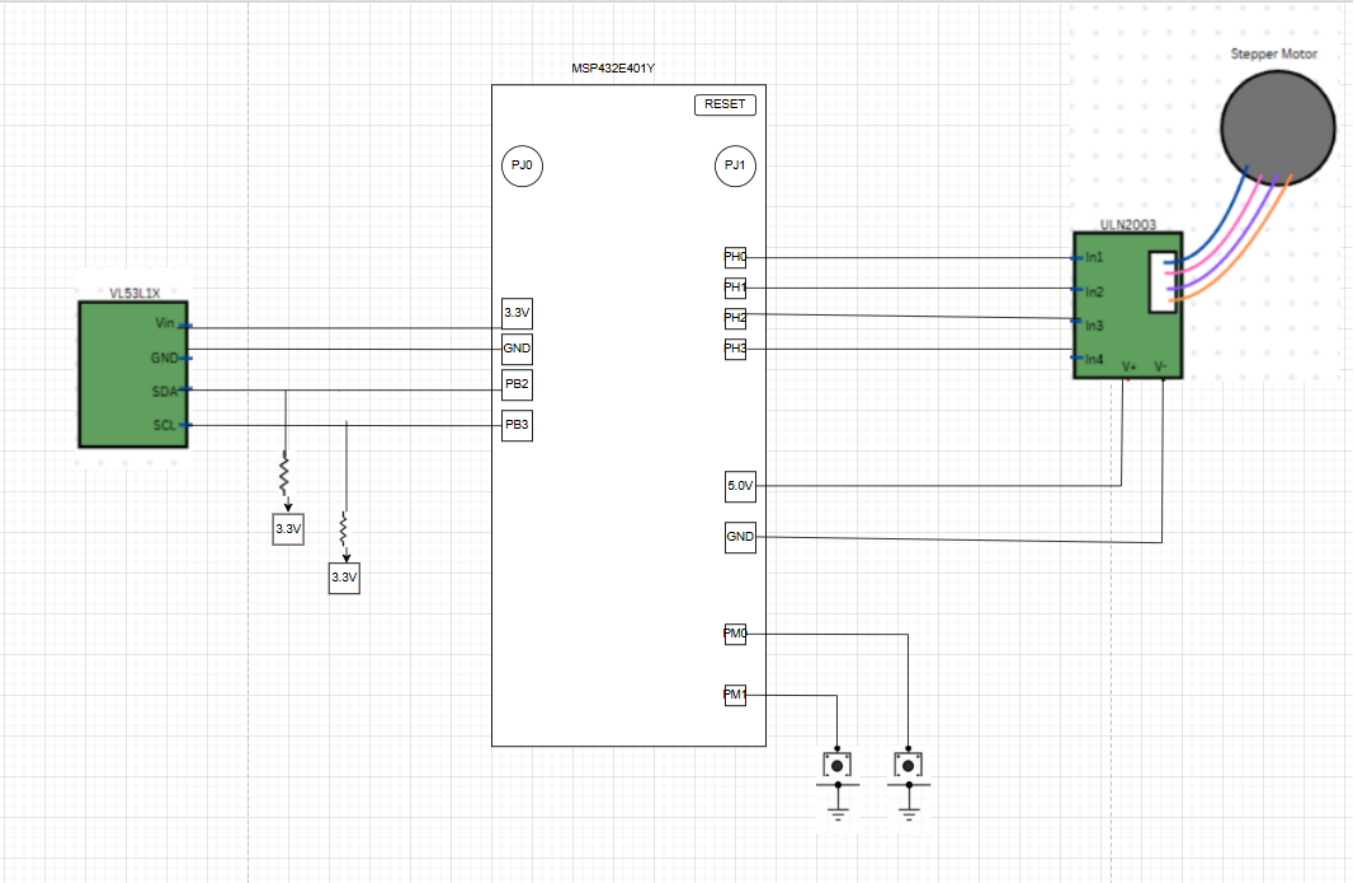
Angular Resolution and Measurement Speed Limitations

The current system is has a resolution of 11.25° for each measurement, which represents a compromise between being able to achieve spatially resolved measurements (spatial detail) and having acceptable system performance. Whether or not more angular precision (i.e., taking measurements at smaller angular steps) provides higher resolution output of the reconstructed 3D model depends on the number of measurements that can be made per scan. As the step size increases, the number of total samples per scan also increases, thus increasing the required scan time and processing demands. Additionally, each sample taken by the Timeof-Flight sensor has an inherent acquisition time which includes some time delay for (i) preparing the data for processing and (ii) processing data since the previous measurement was taken. The cumulative amount of time required for running each measurement, combined with the motor's step and stabilization times, limits how often measurements can be made. Creating an Angular Precision greater than 11.25° may not be a viable option since doing so would limit system responsiveness and ultimately decrease the overall efficiency of the scanning process.

Primary Limitation Element: The ToF sensor measurement and data-ready latency is the dominant factor limiting system speed, as each distance reading requires a non-negligible acquisition time that accumulates across all scan points.

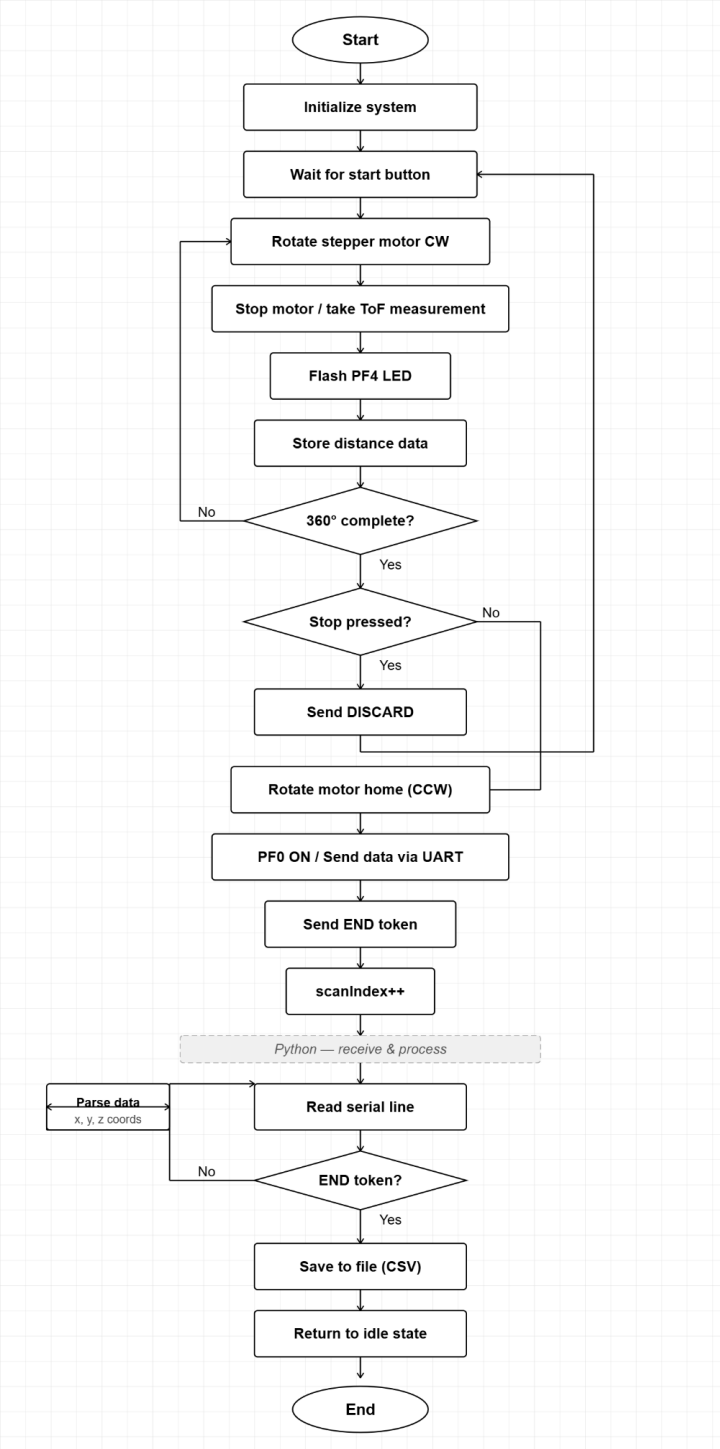
6.0 - Circuit Schematic

Figure 5.0 - Schematic



7.0 - Flow Chart

Figure 6.0 - Flow Chart



More Detialed Flow Chart:
https://drive.google.com/file/d/1wTODCk7TEbljcGIAdvo6eoGTYM1eCXMN/view?usp=sharing
(Please let the document load to view it clearly)

8.0 - References

[1] NASA, “ICESat-2: Measuring Earth’s Ice,” Available: https://www.nasa.gov/mission_pages/icesat-2 . [Accessed: Apr. 4, 2026].

[2] NASA, “Mars Orbiter Laser Altimeter (MOLA),” Available: https://science.nasa.gov/missions/mars-global-surveyor/mola/ . [Accessed: Apr. 4, 2026].

[3] Tesla, Inc., “Autopilot and Full Self-Driving Capability,” Available: https://www.tesla.com/autopilot . [Accessed: Apr. 4, 2026].

[4] US Geological Survey, “What is Lidar?,” Available: https://www.usgs.gov/core-science-systems/ngp/3dep/what-is-lidar . [Accessed: Apr. 4, 2026].